

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №7
По дисциплине «Современные платформы программирования»**

Выполнил:
Студент 3 курса
Группы ПО-12
Пашкевич В. В.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений.

Задание 1

Построение графических примитивов и надписей

Вариант 6

Задать движение окружности по форме так, чтобы при касании границы окружность отражалась от нее.

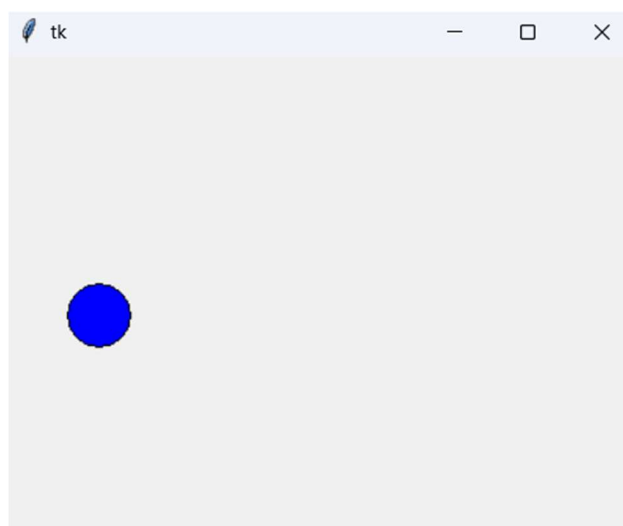
Решение

```
import tkinter as tk

class BouncingCircle:
    def __init__(self, canvas):
        self.canvas = canvas
        self.r = 20
        self.x, self.y = 50, 50
        self.dx, self.dy = 5, 3 # Скорость
        self.circle = self.canvas.create_oval(self.x-self.r, self.y-self.r,
                                                self.x+self.r, self.y+self.r,
                                                fill="blue")
        self.running = True

    def move(self):
        if not self.running: return
        self.canvas.move(self.circle, self.dx, self.dy)
        pos = self.canvas.coords(self.circle)
        # Проверка границ
        if pos[0] <= 0 or pos[2] >= 400:
            self.dx = -self.dx
        if pos[1] <= 0 or pos[3] >= 300:
            self.dy = -self.dy
        self.canvas.after(20, self.move)

root = tk.Tk()
canvas = tk.Canvas(root, width=400, height=300)
canvas.pack()
app = BouncingCircle(canvas)
app.move()
root.mainloop()
```



Задание 2

Реализовать построение заданного типа фрактала по варианту

Вариант 3

Треугольная салфетка Серпинского

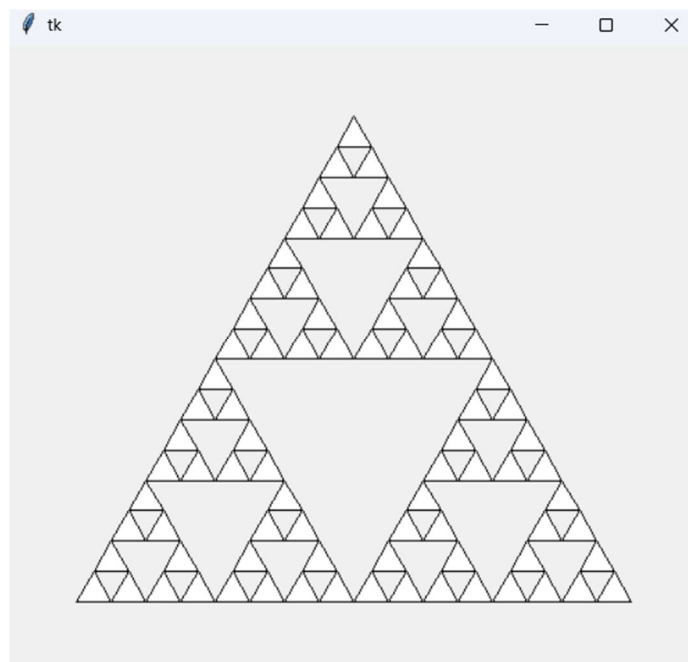
Решение

```
import tkinter as tk

def sierpinski(cvs, p1, p2, p3, depth):
    if depth == 0:
        cvs.create_polygon(p1, p2, p3, outline='black', fill='white')
    else:
        # Находим середины сторон
        p12 = [(p1[0]+p2[0])/2, (p1[1]+p2[1])/2]
        p23 = [(p2[0]+p3[0])/2, (p2[1]+p3[1])/2]
        p31 = [(p3[0]+p1[0])/2, (p3[1]+p1[1])/2]
        # Рекурсивный вызов для 3-х внешних треугольников
        sierpinski(cvs, p1, p12, p31, depth-1)
        sierpinski(cvs, p2, p12, p23, depth-1)
        sierpinski(cvs, p3, p31, p23, depth-1)

root = tk.Tk()
canvas = tk.Canvas(root, width=500, height=450)
canvas.pack()

# Вершины треугольника
points = [[250, 50], [50, 400], [450, 400]]
sierpinski(canvas, points[0], points[1], points[2], 4)
root.mainloop()
```



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.